

Exercício de Fixação — TADs e Orientação a Objetos

Página / Slide 21

Aluno: Rodrigo Del Padre

RA: 24042092-2

Curso: Engenharia de Software (6º Semestre)

Turma: B

Disciplina: Paradigmas de Programação

Referência: R. Sebesta, 11ª ed., Caps. 11 e 12

Trecho 1 — Java

Código analisado:

```
class Animal {
    String som() { return "..."; }
}
class Cachorro extends Animal {
    String som() { return "au"; }
}
Animal x = new Cachorro();
System.out.println(x.som());
```

(a) Qual é a saída?

au

(b) A decisão foi tomada na compilação ou na execução?

Decisão tomada na EXECUÇÃO (Runtime)

Conceito Teórico: Vinculação Dinâmica (*Late Binding / Dynamic Method Dispatch*), Polimorfismo de Subtipagem e Tabela de Métodos Virtuais (*vtable*) (Sebesta, 11ª ed., §12.2.3, p. 492–493 e §12.5, p. 526–529; Slides 15, 17 e 20).

- **Tipo Estático vs. Dinâmico:** A variável polimórfica `x` possui tipo estático `Animal`, mas aponta para uma instância cujo tipo dinâmico no *heap* é `Cachorro`.
- **Métodos Virtuais por Padrão:** Em Java, métodos de instância são inerentemente virtuais. O compilador emite a instrução de bytecode `invokevirtual`, delegando a seleção do método final para a JVM.
- **Despacho via *vtable*:** Durante a execução, a JVM consulta a tabela virtual (*vtable*) anexada ao Registro de Instância de Classe (RIC) do objeto real. O ponteiro de `som()` na *vtable* de `Cachorro` referencia a implementação sobrescrita `Cachorro.som()`.

Trecho 2 — C++

Código analisado:

```
struct A {
    void f() { cout << "A"; }
};
struct B : A {
```

```
void f() { cout << "B"; }  
};  
B b;  
A *p = &b;  
p->f();
```

(a) Qual é a saída?

A

(b) A decisão foi tomada na compilação ou na execução?

Decisão tomada na COMPILAÇÃO (Compile-time)

Conceito Teórico: Vinculação Estática por Padrão (*Static Binding / Early Binding*) e Filosofia do Custo Zero de Abstração (Sebesta, 11ª ed., §12.3.5, p. 497; Slides 17 e 20).

- **Filosofia de C++ (Zero-Overhead Principle):** C++ não impõe o custo de despacho dinâmico (tabelas virtuais e chamadas indiretas) a menos que explicitamente solicitado pelo programador através da palavra-chave `virtual`.
- **Ausência de virtual:** Como o método `f()` em `struct A` não é virtual, nenhuma `vtable` é associada à classe.
- **Resolução pelo Tipo Estático do Ponteiro:** O compilador analisa a chamada `p->f()` observando unicamente o tipo estático do ponteiro `p`, que é `A*`. Ele vincula a chamada diretamente ao endereço de `A::f()` em tempo de compilação.
- **Ocultação (*Name Hiding*):** O método `B::f()` apenas oculta `A::f()` dentro de seu escopo, sem polimorfismo. Como o tipo do ponteiro é `A*`, o método executado é `A::f()`, imprimindo A.